Mustafa Abbas Abokhadher

2240002889

7MA2

Lab 7(A) - Laplacian Filter

In [2]:
```
import cv2
import numpy as np

from IPython.display import Image, display
from matplotlib import pyplot as plt

plt.rc('text', usetex=False)
```

Apply Laplacian Filter on Image to get Sharp Image

In [3]:
```
# ================ ( Laplacian Filter ) ===================


# 0. Image file path
imgFile = "moon.jpg"

# 1. Load image and convert to grayscale
img = cv2.imread(imgFile, 0)

# 2. Manually define the Laplacian Kernel
# This is the standard 3x3 kernel that detects horizontal and vertical edges
manual_kernel = np.array([
    [0,  1,  0],
    [1, -4,  1],
    [0,  1,  0]
], dtype=np.float32)

# Alternative kernel: Detects diagonal edges as well
# manual_kernel = np.array([
#     [1,  1,  1],
#     [1, -8,  1],
#     [1,  1,  1]
# ], dtype=np.float32)

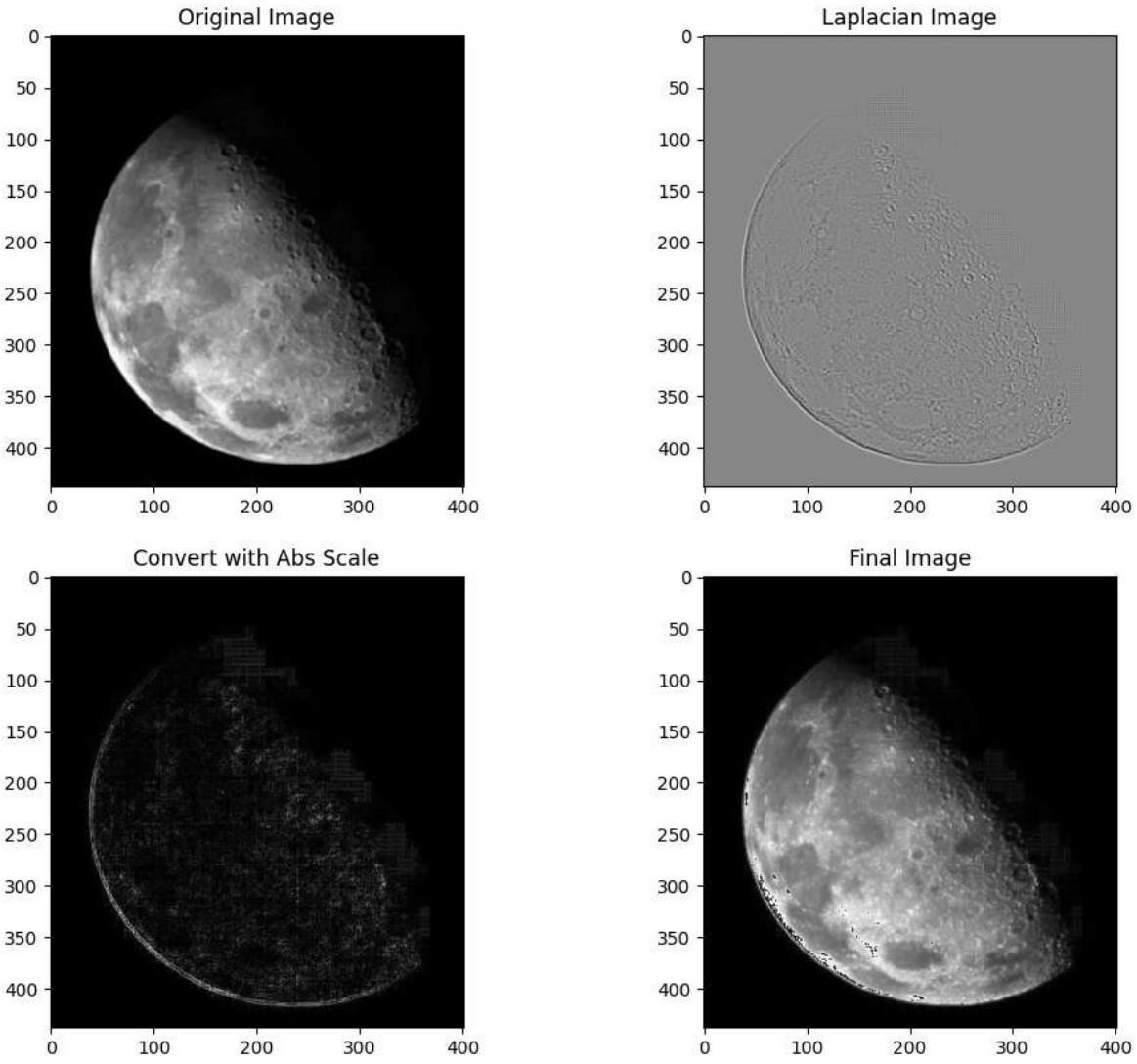
# 3. Apply the filter using filter2D
# -1 indicates the output image will have the same depth as the source
# However, for Laplacian, use cv2.CV_64F to avoid clipping negative values
laplacian_manual = cv2.filter2D(img, cv2.CV_64F, manual_kernel)

# 4. Take absolute value and convert back to 8-bit (0-255)
```

```
laplacian_abs = cv2.convertScaleAbs(laplacian_manual)

finalImage = img + laplacian_abs

# 5. Display images
plt.figure(figsize=(12, 10))
plt.subplot(221), plt.imshow(img, cmap='gray'), plt.title('Original Image')
plt.subplot(222), plt.imshow(laplacian_manual, cmap='gray'), plt.title('Laplacia
plt.subplot(223), plt.imshow(laplacian_abs, cmap='gray'), plt.title('Convert wit
plt.subplot(224), plt.imshow(finalImage, cmap='gray'), plt.title('Final Image')
plt.show()
```



In [4]:

```
# ================== ( Laplacian Filter ) ===============

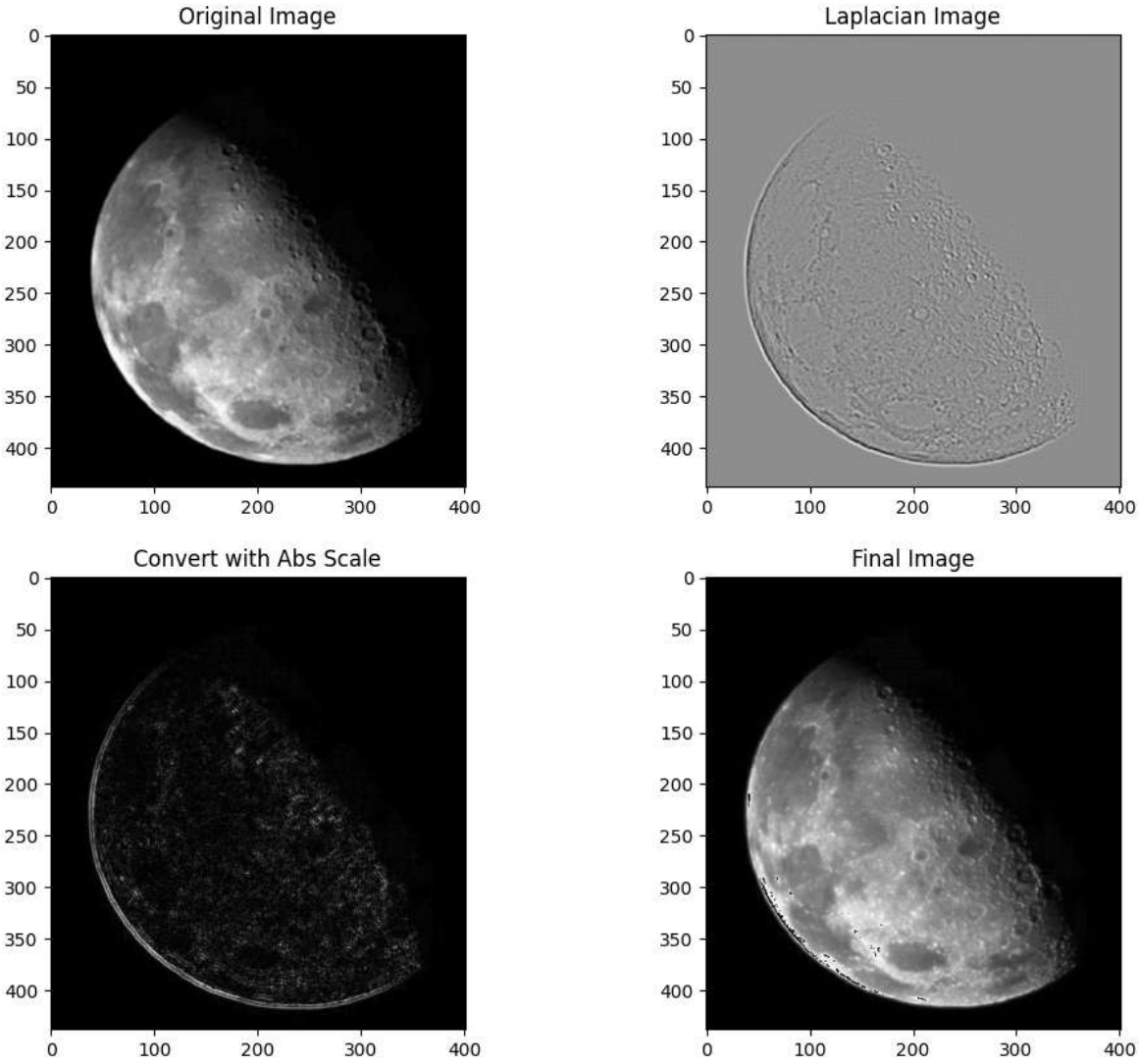
# 1. Optional: Apply Gaussian Blur to reduce noise
# This prevents the algorithm from picking up tiny details as edges
blurred = cv2.GaussianBlur(img, (3, 3), 0)

# 2. Apply the Laplacian Algorithm
# Use CV_64F to capture both positive and negative gradients (essential for edge
laplacian = cv2.Laplacian(blurred, cv2.CV_64F)

# 3. Convert back to 8-bit image to view it properly
laplacian_8bit = cv2.convertScaleAbs(laplacian)

# 4. Adding Laplace and origianl Image
finalImage2 = img + laplacian_8bit
```

```
# 5. Display images
plt.figure(figsize=(12, 10))
plt.subplot(221), plt.imshow(img, cmap='gray'), plt.title('Original Image')
plt.subplot(222), plt.imshow(laplacian, cmap='gray'), plt.title('Laplacian Image
plt.subplot(223), plt.imshow(laplacian_8bit, cmap='gray'), plt.title('Convert wi
plt.subplot(224), plt.imshow(finalImage2, cmap='gray'), plt.title('Final Image')
plt.show()
```



In [5]:
```
# ============ ( Solution: Proper Laplacian Sharpening ) ============
f = img.astype(np.float64)

# 4-neighbour kernel (centre = -4)  and  8-neighbour kernel (centre = -8)
k4 = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]], dtype=np.float64)
k8 = np.array([[1, 1, 1], [1, -8, 1], [1, 1, 1]], dtype=np.float64)

lap4 = cv2.filter2D(f, cv2.CV_64F, k4)
lap8 = cv2.filter2D(f, cv2.CV_64F, k8)

# Negative centre  ->  g = f - laplacian   (keep sign, work in float, then clip)
sharp4 = np.clip(f - lap4, 0, 255).astype(np.uint8)
sharp8 = np.clip(f - lap8, 0, 255).astype(np.uint8)

# Same result using the built-in function
lap_cv = cv2.Laplacian(f, cv2.CV_64F, ksize=1)       # ksize=1 -> the [0 1 0; 1
```

```
sharp_cv = np.clip(f - lap_cv, 0, 255).astype(np.uint8)
print("cv2.Laplacian == manual 4-neighbour kernel:", np.allclose(lap_cv, lap4))

plt.figure(figsize=(15, 10))
plt.subplot(231), plt.imshow(img, cmap='gray'),     plt.title('Original Image')
plt.subplot(232), plt.imshow(lap4, cmap='gray'),    plt.title('Laplacian (4-neig
plt.subplot(233), plt.imshow(sharp4, cmap='gray'),  plt.title('Sharpened: f - La
plt.subplot(234), plt.imshow(lap8, cmap='gray'),    plt.title('Laplacian (8-neig
plt.subplot(235), plt.imshow(sharp8, cmap='gray'),  plt.title('Sharpened: f - La
plt.subplot(236), plt.imshow(sharp_cv, cmap='gray'),plt.title('Sharpened with cv
for ax in plt.gcf().axes: ax.axis('off')
plt.tight_layout()
plt.show()
```

```
cv2.Laplacian == manual 4-neighbour kernel: True
```

